

DSA & Advanced DSA Roadmap

Phase 8 — Sorting Algorithms

Track 2: Core Algorithms

This phase is implementation-heavy by design: 7 classical sorting algorithms must be built from scratch, compared, and benchmarked. Only 3 named patterns follow, each backed by verified problems that genuinely combine sorting with another technique (custom comparators, greedy two-pointer matching, or interval scheduling).

Learning Objectives

By the end of this phase, learners will be able to:

- Implement Bubble, Selection, Insertion, Merge, Quick, Heap, and Counting Sort from scratch.
- State the time and space complexity of each algorithm, including best, average, and worst case where they differ.
- Explain stability in sorting (whether equal elements retain their relative order) and identify which of the 7 algorithms are stable.
- Choose the appropriate sorting algorithm for a given constraint (limited memory, nearly-sorted input, bounded integer range, need for stability).
- Apply custom comparators to sort by derived criteria, and recognize when sorting is a useful first step before a greedy or interval-based strategy.

Implementation & Internals

Each algorithm below should be implemented from scratch, then tested against a library sort to confirm correctness on: empty array, single element, all-duplicates, already-sorted, and reverse-sorted inputs.

Bubble Sort

Time: $O(n^2)$ average/worst, $O(n)$ best (already sorted) | Space: $O(1)$

- Repeatedly swaps adjacent out-of-order elements until no swaps are needed in a full pass.
- Add an early-exit flag: if a full pass makes no swaps, the array is sorted — this gives the $O(n)$ best case.
- Stable: equal elements are never swapped past each other.

Selection Sort

Time: $O(n^2)$ in all cases | Space: $O(1)$

- Repeatedly selects the minimum remaining element and places it at the next sorted position.
- Number of swaps is $O(n)$ even though comparisons remain $O(n^2)$ — useful when writes are expensive.

- Not stable: the minimum element is swapped into a position that may jump over equal elements.

Insertion Sort

Time: $O(n^2)$ average/worst, $O(n)$ best (already sorted) | Space: $O(1)$

- Builds the sorted portion one element at a time by inserting each new element into its correct position.
- Stable: equal elements are inserted without swapping past each other.
- The algorithm of choice for small or nearly-sorted inputs; used as the base case in hybrid sorts like Timsort.

Merge Sort

Time: $O(n \log n)$ in all cases | Space: $O(n)$

- Recursively splits the array in half, sorts each half, then merges the two sorted halves.
- Stable by construction — the merge step preserves relative order of equal elements.
- $O(n)$ auxiliary space cost; in-place merge sort variants trade simplicity for reduced memory.

Quick Sort

Time: $O(n \log n)$ average, $O(n^2)$ worst (sorted input with naive pivot) | Space: $O(\log n)$ average (recursion stack)

- Selects a pivot, partitions the array into elements less than and greater than the pivot, then sorts each side recursively.
- A fixed first/last-element pivot degrades to $O(n^2)$ on sorted or reverse-sorted input; randomized or median-of-three pivot selection avoids this.
- Not stable in standard implementations.

Heap Sort

Time: $O(n \log n)$ in all cases | Space: $O(1)$

- Builds a max-heap from the array in $O(n)$ via heapify, then repeatedly extracts the maximum and rebuilds.
- Connects directly to Phase 12 (Heap): heap sort is the natural bridge between sorting and the heap data structure.
- Not stable.

Counting Sort

Time: $O(n + k)$, where k is the range of input values | Space: $O(k)$

- Counts occurrences of each distinct value, then reconstructs the sorted array from those counts — no comparisons involved.
- Only works efficiently when input is integers within a known, bounded range; not a general-purpose sort.
- Stable when implemented correctly (iterate the count array in order and place from right to left).

Implementation Exercise

- After implementing all 7, benchmark them against each other on random, sorted, reverse-sorted, and all-duplicate inputs of increasing size (e.g. $n = 100, 1,000, 10,000$). Produce a short comparison table of measured running times.
 - Identify which of your 7 implementations are stable by running a test case with records that share a sort key but differ in a secondary field — stable sorts must preserve their original relative order.
-

Pattern Mastery

Pattern 1: Custom Sorting

Concepts

- Sorting by a derived key rather than natural element order, using a custom comparator.
- Recognizing when natural numeric or lexicographic order gives the wrong answer and a problem-specific comparator is required.

Practice Problems

- Sort a string's characters in descending order of frequency.
- Given a list of non-negative integers, arrange them to form the largest possible concatenated number.

LeetCode

- [Sort Characters By Frequency — leetcode.com/problems/sort-characters-by-frequency](https://leetcode.com/problems/sort-characters-by-frequency)
- [Largest Number — leetcode.com/problems/largest-number](https://leetcode.com/problems/largest-number)

Pattern 2: Sorting + Greedy

Concepts

- Sorting the input first so that a subsequent greedy, single-pass decision becomes correct and simple.
- The two-pointer-after-sort shape: process smallest and largest remaining elements together to make a locally optimal, irreversible choice.

Practice Problems

- Maximize the number of content children by matching the smallest sufficient cookie to each child.
- Find the minimum number of boats needed to carry all people, pairing the heaviest remaining person with the lightest who still fits.

LeetCode

- [Assign Cookies — leetcode.com/problems/assign-cookies](https://leetcode.com/problems/assign-cookies)
- [Boats to Save People — leetcode.com/problems/boats-to-save-people](https://leetcode.com/problems/boats-to-save-people)

Pattern 3: Interval Sorting

Concepts

- Sorting intervals by start time (to merge or detect overlaps left-to-right) or by end time (to greedily select the maximum number of non-conflicting intervals).
- The sort key choice (start vs. end) depends on the objective: merging requires sort-by-start; maximum non-overlapping selection requires sort-by-end.

Practice Problems

- Find the minimum number of conference rooms required given a list of meeting time intervals.
- Find the minimum number of intervals to remove so the rest do not overlap.
- Find the minimum number of arrows needed to burst all balloon intervals.

LeetCode

- [Meeting Rooms II — leetcode.com/problems/meeting-rooms-ii](https://leetcode.com/problems/meeting-rooms-ii)
- [Non-overlapping Intervals — leetcode.com/problems/non-overlapping-intervals](https://leetcode.com/problems/non-overlapping-intervals)
- [Minimum Number of Arrows to Burst Balloons — leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons](https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons)

Note: Meeting Rooms I (LeetCode 252) is Premium-only and not assigned here. Meeting Rooms II is free and covers the same room-counting concept at greater depth.

Phase Assessment

Implementation Tasks

- Implement all 7 sorting algorithms from scratch: Bubble, Selection, Insertion, Merge, Quick, Heap, and Counting Sort.
- Benchmark all 7 on the same random, sorted, reverse-sorted, and all-duplicate inputs of increasing size. Produce a comparison table of measured times.
- Identify which implementations are stable and verify with a secondary-field test case.
- Deliberately test Quick Sort on sorted input with a naive pivot, observe the $O(n^2)$ degradation, then fix it with randomized pivot selection.

Recommended LeetCode Target

This phase has 7 LeetCode problems (smaller than the 9–11 problem range of adjacent phases) because the bulk of its workload is the 7 from-scratch implementations and benchmarking exercise above. Every link is verified free and cross-checked against all other phases to guarantee no repeats.

Easy Problems

- leetcode.com/problems/sort-characters-by-frequency
- leetcode.com/problems/assign-cookies

Medium Problems

Custom Sorting

- leetcode.com/problems/largest-number

Sorting + Greedy

- leetcode.com/problems/boats-to-save-people

Interval Sorting

- leetcode.com/problems/meeting-rooms-ii
- leetcode.com/problems/non-overlapping-intervals
- leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons

On scope

No Hard tier this phase. The natural Hard sorting problems (e.g. Count of Smaller Numbers After Self) require a Binary Indexed Tree, which belongs to Phase 17. The smaller LeetCode count is intentional: the from-scratch algorithm implementations and benchmarking exercise are this phase's primary assessment deliverable.

Coding Challenges (Assessment Day)

- Implement Quick Sort and demonstrate its $O(n^2)$ worst case on adversarial (sorted) input, then fix with randomized pivot.
- Largest Number

- Meeting Rooms II
 - Non-overlapping Intervals
 - Boats to Save People
-

Expected Outcome

Students should be able to:

- Implement all 7 classical sorting algorithms from scratch and state their time and space complexity from memory.
- Choose the right sorting algorithm for a given constraint: memory limits, stability requirements, nearly-sorted input, or bounded integer range.
- Recognize sorting-adjacent patterns: custom comparators, sort-then-greedy, and interval sorting with the correct key.
- Solve unseen sorting-pattern problems using pattern-based thinking rather than memorized solutions.

Once this phase is approved, Phase 9 (Recursion) follows, continuing Track 2: Core Algorithms.